

Lab 4 - Memory-Efficient Transformer Training Techniques

Bartosz Gacek

1. Wprowadzenie

Celem tego zadania jest porównanie technik optymalizacji pamięci wykorzystywanych podczas trenowania modeli Transformer. W ramach eksperymentu zaimplementowano i porównano cztery podejścia:

1. **Baseline (FP32/TF32)** - bazowy model z `batch_size = 128`
2. **BF16 Mixed Precision** - automatyczna precyzja BF16
3. **FlashAttention** - standardowa implementacja FlashAttention v2
4. **Windowed (Local) Attention** - lokalny mechanizm uwagi
5. **Gradient Checkpointing** - gradient checkpointing z biblioteki pytorch

W przeciwieństwie do poprzednich laboratoriów, gdzie proces uczenia nie był oparty na epokach, specyfika tego zadania wymusiła modyfikację skryptu treningowego. Ze względu na dużą liczbę wymaganych eksperymentów, zdecydowałem się ograniczyć trening do jednej epoki.

2. Konfiguracja eksperymentów

2.1 Zbiór Danych

Ten sam zbiór danych oraz tokenizer, które zostały użyte w zadaniu 1.

- **Źródło:** Wolne Lektury Corpus (z Lab 1)
- **Rozmiar:** 26,741,355 tokenów
- **Podział:** 90% trening, 10% walidacja
- **Tokenizer:** tiktoken (cl100k_base)

2.2 Architektura Modelu

Decoder-Only Transformer:

- **Liczba parametrów:** 87.80M
- **Liczba warstw:** 6
- **Rozmiar embeddingu:** 384
- **Liczba attention heads:** 4
- **Dropout:** 0.2
- **Długość sekwencji:** 128 tokenów

2.3 Hiperparametry Trenowania

- **Optimizer:** AdamW
- **Learning rate:** $3e-4$

- **Liczba epok:** 1
- **Bazowy Batch size:** 128

2.4 Sprzęt

- **GPU:** NVIDIA A40
- **Pamięć HBM:** ~48GB
- **Framework:** PyTorch 2.5 + CUDA 12.4

Ze względu na liczne problemy z kompatybilnością bibliotek oraz długi czas kompilacji flash-attention w środowisku lokalnym, zdecydowałem się na skorzystanie z gotowych, prekompilowanych pakietów (wheels). Wykorzystanie repozytorium `mjun0812/flash-attention-prebuild-wheels` pozwoliło na natychmiastową integrację mechanizmu FlashAttention bez potrzeby walczenia z błędami kompilacji.

3. Wyniki Eksperymentów

3.1 Tabela Porównawcza - Batch Size 128 (Bazowy)

Method	Batch Size	Peak Memory (GB)	Sec/Step	Epoch Time (min)	Val Perplexity	Memory Reduction	Speedup
Baseline (FP32)	128	27.05	0.6292	17.11	60.06	-	-
BF16	128	21.02	0.4157	11.31	59.96	22.3%	34.0%
FlashAttention	128	20.73	0.3360	9.15	60.66	23.4%	46.6%
Windowed (w=64)	128	20.03	0.3590	9.76	59.41	25.9%	42.9%
Grad. Checkpoint	128	25.01	0.6542	17.76	59.95	7.5%	-4.0%

3.2 Tabela Porównawcza - Maksymalny Batch Size

Maksymalny znaleziony batch size, gdzie nie dostawałem błędu out-of-memory

Method	Max Batch Size	BS Increase	Peak Memory (GB)	Sec/Step	Val Perplexity
Baseline (FP32)	128	1.00×	27.05	0.6292	60.06
BF16	176	1.38×	28.70	0.5648	63.70
FlashAttention	224	1.75×	35.95	0.5801	67.84
Windowed (w=64)	224	1.75×	28.49	0.4740	64.97
Grad. Checkpoint	176	1.38×	34.26	0.9011	92.71*

* Uwaga: Wysoka wartość perplexity sugeruje problemy ze stabilnością trenowania przy większym batch size

3.3 Wykorzystanie pamięci w poszczególnych etapach (BS=128)

Method	Forward Pass (GB)	Backward Pass (GB)	Peak Memory (GB)
Baseline	14.83	27.05	27.05
BF16	14.94	21.02	21.02
FlashAttention	13.89	20.73	20.73
Windowed	13.95	20.03	20.03
Grad. Checkpoint	12.79	25.01	25.01

4. Analiza i Dyskusja

4.1 Efektywność Pamięciowa

Najwyższą skuteczność w redukcji zapotrzebowania na VRAM wykazały metody Windowed Attention oraz FlashAttention. W przypadku pierwszej z nich zysk wynika bezpośrednio z ograniczenia złożoności obliczeniowej poprzez zawężenie uwagi do lokalnego okna, co radykalnie zmniejsza rozmiar macierzy. FlashAttention uzyskuje zbliżone rezultaty dzięki optymalizacjom niskopoziomowym wykonywane na poziomie programu CUDA.

Zastosowanie BF16 Mixed Precision pozwala na oszczędności dzięki zmianie precyzji typów danych z 4-bajtowych na 2-bajtowe, przy zachowaniu stabilności dzięki akumulacji w formacie FP32. Najmniejszy wpływ na pamięć odnotowano przy Gradient Checkpointing, co wynika z charakterystyki modelu – przy niewielkiej liczbie warstw oszczędność na samych aktywacjach jest ograniczona.

4.2 Wydajność Obliczeniowa

Pod względem szybkości przetwarzania dominuje FlashAttention, co jest efektem synergii zoptymalizowanych kerneli CUDA oraz redukcji bottlenecka, jakim jest przepustowość pamięci GPU. Windowed Attention również oferuje znaczące przyspieszenie, wynikające z mniejszej liczby operacji na krok treningowy i efektywniejszego wykorzystania pamięci cache.

Jedyną techniką odnotowującą spadek wydajności był Gradient Checkpointing. Jest to klasyczny kompromis (trade-off) – oszczędność pamięci zostaje okupiona dodatkowym kosztem obliczeniowym wynikającym z konieczności powtórznego wyznaczania aktywacji podczas fazy wstecznej.

4.3 Jakość Modelu (Perplexity)

Przy standardowym rozmiarze batch size wszystkie techniki zachowują stabilność, a wyniki perplexity pozostają na zbliżonym poziomie. Sugeruje to, że mechanizmy takie jak FlashAttention czy BF16 są w pełni bezpieczne dla jakości predykcji. Lekka poprawa wyniku w przypadku Windowed Attention może wskazywać na pozytywny wpływ regularyzacji kontekstu lokalnego.

Problemy pojawiają się przy skalowaniu do maksymalnego dopuszczalnego batch size. Zaobserwowany wówczas wzrost perplexity wynika z rzadszej aktualizacji wag, co utrudnia zbieżność. Szczególną uwagę zwraca drastyczny spadek jakości przy Gradient Checkpointing; sugeruje on niestabilność numeryczną przy dużych partiach danych lub błąd w doborze hiperparametrów (np. zbyt wysoki learning rate dla tej konfiguracji).

4.4 Skalowalność i Połączenie Technik

Techniki `FlashAttention` oraz `Windowed Attention` oferują największy potencjał skalowania, umożliwiając znaczące zwiększenie rozmiaru `batch size`, co bezpośrednio przekłada się na wyższy `throughput`. Pozostałe metody oferują w tym zakresie bardziej umiarkowane korzyści.

Analizowane rozwiązania można łączyć, aby uzyskać efekt synergii. Para `FlashAttention` i `Gradient Checkpointing` to rozwiązanie dedykowane dla scenariuszy z krytycznym brakiem pamięci, podczas gdy `Windowed Attention` i `BF16` stanowią optymalny balans dla standardowych procesów treningowych. Należy jednak monitorować kumulujący się narzut obliczeniowy oraz potencjalną degradację jakości przy łączeniu wielu technik optymalizacyjnych jednocześnie.

Techniki można łączyć, np. `FlashAttention` + `Gradient Checkpointing` (maksymalna redukcja pamięci), `Windowed Attention` + `BF16` (balans między pamięcią a jakością) czy `FlashAttention` + większy model (wykorzystanie zaoszczędzonej pamięci).

Jednak należy uważać na kumulujące się efekty numeryczne `BF16` + `Gradient Checkpoint`, overhead obliczeniowy (wiele technik razem może być wolniejsze) oraz degradację jakości `Windowed` + agresywne checkpointing.

5. Wnioski

5.1 Rekomendacje

Wybór optymalnej techniki optymalizacji zależy od priorytetów projektowych oraz dostępnych zasobów sprzętowych. Pod kątem maksymalnej wydajności bezdyskusyjnym liderem jest `FlashAttention`, który oferuje najkorzystniejszy balans między redukcją zapotrzebowania pamięci (o 23%) a wzrostem szybkości obliczeń (o 47%). Rozwiązanie to, zachowując pełną jakość modelu, pozwoliło na trenowanie z największym batch size (224).

W scenariuszach cechujących się krytycznym deficytem pamięci najskuteczniejszy okazuje się mechanizm `Windowed Attention`. Zapewnia on najwyższą, 26-procentową oszczędność VRAM przy jednoczesnym przyspieszeniu pracy o 43%, oferując przy tym potencjalną poprawę jakości dzięki regularyzacji lokalnego kontekstu. Z kolei w warunkach produkcyjnych, szczególnie przy mniejszych modelach, warto rozważyć wykorzystanie formatu `BF16`. Mimo ryzyka nieznacznego pogorszenia jakości odpowiedzi, jego głównymi zaletami są prostota implementacji bez konieczności zmian w architekturze oraz zyski wydajnościowe (22% mniej pamięci i 34% szybsze przetwarzanie).

Uzupełniając, w przypadku pracy z bardzo dużymi modelami, warto rozważyć `Gradient Checkpointing`. Należy jednak pamiętać, że jest to technika przeznaczona głównie do ekstremalnych ograniczeń pamięciowych; wymaga ona ostrożności przy skalowaniu `batch_size` i nie zaleca się stosowania jej jako jedynej metody optymalizacji procesu.

5.2 Kluczowe Obserwacje

Analiza wyników pozwala na sformułowanie kilku kluczowych wniosków w zakresie optymalizacji procesów trenowania. `FlashAttention` jawi się jako najbardziej efektywny wybór ogólny, oferujący znaczące korzyści wydajnościowe przy minimalnych kompromisach. Choć proces integracji może być utrudniony przez konieczność manualnej kompilacji, problem ten skutecznie rozwiązuje wykorzystanie gotowych, prekompilowanych repozytoriów. Równie obiecującą, choć często niedocenianą techniką, jest `Windowed Attention`. Wykazano, że przy odpowiednio dobranym oknie (np. 64 tokeny) metoda ta pozwala na istotny zysk w szybkości i oszczędności pamięci bez mierzalnej utraty jakości modelu.

W kontekście standardów operacyjnych, wykorzystanie formatu BF16 jest całkiem dobrym rozwiązaniem. Jest ono bezpieczne i wyjątkowo proste we wdrożeniu; mimo teoretycznego ryzyka spadku jakości odpowiedzi, w przeprowadzonym badaniu problem ten nie wystąpił. Z kolei technika Gradient Checkpointing wymaga szczególnej ostrożności. Zaobserwowany wzrost wskaźnika perplexity przy dużym batch size sugeruje konieczność precyzyjnego tuningu hiperparametrów lub wskazuje na specyficzną charakterystykę wykorzystanego zbioru danych. Podkreśla to jednocześnie ogólną zasadę, że skalowanie parametru batch size nie jest procesem bezkosztowym i każdorazowo wymaga rewizji konfiguracji modelu w celu zachowania stabilności procesu uczenia.

6. Podsumowanie Wyników

Kryterium	Zwycięzca	Wartość
Memory Reduction	Windowed Attention	-25.9%
Speedup	FlashAttention	+46.6%
Max Batch Size	FlashAttention/Windowed	224 (1.75×)
Perplexity	Windowed Attention	59.41
Łatwość implementacji	BF16	Łatwa
Ogólny balans	FlashAttention	Wygrywa w 3/5 kryteriów